06. MoE LLMs - Mixtral & Deepseek-V2 아키텍처 상세 분석

목차

1. MoE (Mixture-of-Experts) 기본 개념
2. Mixtral 아키텍처
3. [MixtralMoE 구현 분석](#3-mixt

ralmoe-구현-분석)
4. FusedMoE 커널
5. Expert 선택 알고리즘
6. Deepseek-V2 MoE
7. Mixtral vs Deepseek-V2 비교
8. Expert Parallelism & Load Balancing
9. 성능 측정 및 최적화
10. 트러블슈팅

---

1. MoE (Mixture-of-Experts) 기본 개념

1.1 MoE란?

Mixture-of-Experts (MoE)는 여러 개의 "전문가(Expert)" 네트워크를 사용하여 모델의 capacity를 늘리면서도 실제 계산량은 줄이는 기법입니다.

전통적인 Dense Model vs MoE

```
┌──────────────────────────────────────────────────────────────┐
│                    Dense Model (Llama)                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
```

```
│  Input (hidden_size)                                    │
│    │                                                     │
│    ├─► gate_proj ──┐                                    │
│    │               ├─► SwiGLU ──► down_proj ──► Output   │
│    └─► up_proj ────┘                                    │
│                                                          │
│  모든 파라미터가 모든 토큰에 사용됨                       │
│  Computation: O(hidden_size × intermediate_size)        │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│                    MoE Model (Mixtral)                   │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Input (hidden_size)                                    │
│    │                                                     │
│    ├─► Router (Gate) ──► Top-K Selection                │
│    │                                                     │
│    ├─► Expert 0 ──┐                                     │
│    ├─► Expert 1   │                                     │
│    ├─► Expert 2   ├─► Weighted Sum ──► Output           │
│    ├─► ...        │                                     │
│    └─► Expert 7 ──┘                                     │
│                                                          │
│  각 토큰은 Top-K개의 Expert만 사용                       │
│  Computation: O(hidden_size × intermediate_size × K / N)│
│  (N = 총 Expert 수, K = 활성화 Expert 수)                │
└──────────────────────────────────────────────────────────┘
```

1.2 MoE의 핵심 요소

(1) Router (Gate Network)

```
# vllm/model_executor/models/mixtral.py:121-128
self.gate = ReplicatedLinear(
    hidden_size,
    num_experts,  # Mixtral-8x7B: 8개 experts
    bias=False,
    params_dtype=params_dtype,
    quant_config=None,
    prefix=f"{prefix}.gate",
)
```

역할:

- 입력 토큰을 분석하여 어떤 Expert에게 보낼지 결정
- Softmax 적용하여 확률 분포 생성
- Top-K개의 Expert 선택

출력:

```
router_logits: [num_tokens, n_experts]  # 각 expert에 대한 점수
# Mixtral: [batch_size * seq_len, 8]
```

(2) Experts

```
# vllm/model_executor/layers/fused_moe/fused_moe.py:130-144
self.experts = FusedMoE(
    num_experts=num_experts,      # 8 (Mixtral)
    top_k=top_k,                   # 2 (Mixtral)
    hidden_size=hidden_size,       # 4096
    intermediate_size=intermediate_size,  # 14336
    params_dtype=params_dtype,
    reduce_results=True,
    renormalize=True,
    quant_config=quant_config,
    tp_size=tp_size,
    prefix=f"{prefix}.experts",
)
```

구조:

- 각 Expert는 독립적인 Feed-Forward Network (FFN)
- 구조는 Llama의 MLP와 동일 (gate_proj, up_proj, down_proj)
- Expert마다 다른 파라미터를 학습

(3) Top-K Selection

```
예시: Mixtral-8x7B (8 experts, top_k=2)

Input token: "machine"

Router scores:
  Expert 0: 0.05
  Expert 1: 0.12
```

```
  Expert 2: 0.31  ← Top-1 (선택)
  Expert 3: 0.08
  Expert 4: 0.25  ← Top-2 (선택)
  Expert 5: 0.03
  Expert 6: 0.11
  Expert 7: 0.05

Normalized weights (Softmax on Top-2):
  Expert 2: 0.65  (0.31 / (0.31 + 0.25))
  Expert 4: 0.35  (0.25 / (0.31 + 0.25))

Output = 0.65 × Expert2(input) + 0.35 × Expert4(input)
```

1.3 MoE의 장점

(1) 파라미터 효율성

```
Dense Model (Llama-70B):
  - Total params: 70B
  - Active params per token: 70B
  - Computation: Full model

MoE Model (Mixtral-8x7B):
  - Total params: 47B (8 experts × 7B each, shared attention)
  - Active params per token: ~13B (2 experts × 7B each + shared)
  - Computation: ~1/4 of equivalent dense model

→ 70B Dense와 비슷한 성능을 13B active params로 달성
```

(2) 전문화 (Specialization)

- 각 Expert가 특정 도메인/태스크에 특화
- 예시:
    - Expert 0: 수학/과학
    - Expert 1: 코드/프로그래밍
    - Expert 2: 자연어/대화
    - Expert 3: 추론/논리
    - ...

(3) 확장성 (Scalability)

```
# Expert 수를 늘려도 active computation은 일정
Mixtral-8x7B:  top_k=2, active=14B
Mixtral-16x7B: top_k=2, active=14B  (동일!)

# 더 큰 모델 capacity, 같은 inference cost
```

1.4 MoE의 도전 과제

(1) Load Imbalance

```
이상적인 분포:
  Expert 0: ████████ 12.5%
  Expert 1: ████████ 12.5%
  Expert 2: ████████ 12.5%
  ...
  Expert 7: ████████ 12.5%

실제 분포 (문제):
  Expert 0: ═══════════════ 35%  ← Overused
  Expert 1: ═══ 5%
  Expert 2: ═══════ 15%
  Expert 3: ═══════ 10%
  Expert 4: ═══ 8%
  Expert 5: ═══════ 12%
  Expert 6: ═══════ 10%
  Expert 7: ═══ 5%                ← Underused

문제:
- Expert 0이 병목 현상 발생
- Expert 1, 7은 낭비
- 전체 throughput 감소
```

해결책:

- Load balancing loss 추가
- Expert capacity 제한
- Random routing noise

(2) Communication Overhead

```
# Expert Parallelism (EP) 사용 시
# GPU 0: Expert 0, 1
```

```
# GPU 1: Expert 2, 3
# GPU 2: Expert 4, 5
# GPU 3: Expert 6, 7

# All-to-All communication 필요
Token → GPU 0 → [Expert 2 필요] → GPU 1 (통신!)
                [Expert 4 필요] → GPU 2 (통신!)
```

(3) 메모리 사용량

```
Dense Model (Llama-70B):
  GPU Memory: ~140GB (FP16)

MoE Model (Mixtral-8x7B):
  GPU Memory: ~94GB (FP16)
  - 8 experts × 7B = 56B params
  - Shared layers (attention 등) = 13B params
  - Total = 69B params → ~138GB

하지만:
  - Active memory: ~26GB (2 experts + shared)
  - vLLM의 paged attention으로 KV cache 효율적 관리
```

---

2. Mixtral 아키텍처

2.1 Mixtral 모델 개요

Mixtral-8x7B:

- **Released**: 2023년 12월 (Mistral AI)
- **Architecture**: Sparse MoE Transformer
- **Experts**: 8개
- **Top-K**: 2개
- **Total params**: 47B
- **Active params**: ~13B per token
- **Context length**: 32K tokens

Mixtral vs Llama 비교:

```
# Llama-70B
LlamaDecoderLayer:
  ├─ Self-Attention
  └─ MLP (Feed-Forward)
      ├─ gate_proj: [4096, 14336]
      ├─ up_proj:   [4096, 14336]
      └─ down_proj: [14336, 4096]

# Mixtral-8x7B
MixtralDecoderLayer:
  ├─ Self-Attention (동일)
  └─ MoE (Sparse)
      ├─ Router: [4096, 8]
      └─ 8 Experts (각각 Llama MLP와 동일)
          ├─ Expert 0: MLP
          ├─ Expert 1: MLP
          ├─ ...
          └─ Expert 7: MLP
```

2.2 MixtralDecoderLayer 구조

```python
# vllm/model_executor/models/mixtral.py:240-298
class MixtralDecoderLayer(nn.Module):
    def __init__(
        self,
        config: MixtralConfig,
        cache_config: CacheConfig | None = None,
        quant_config: QuantizationConfig | None = None,
        prefix: str = "",
        enable_eplb: bool = False,
    ) -> None:
        super().__init__()
        self.hidden_size = config.hidden_size

        # RoPE theta
        rope_theta = getattr(config, "rope_theta", 10000)

        # Self-Attention (Llama와 동일)
        self.self_attn = MixtralAttention(
            config=config,
            hidden_size=self.hidden_size,
            num_heads=config.num_attention_heads,
            max_position=config.max_position_embeddings,
            num_kv_heads=config.num_key_value_heads,
```

```
        rope_theta=rope_theta,
        cache_config=cache_config,
        quant_config=quant_config,
        prefix=f"{prefix}.self_attn",
    )

    # MoE (Mixtral의 핵심!)
    self.block_sparse_moe = MixtralMoE(
        num_experts=config.num_local_experts,      # 8
        top_k=config.num_experts_per_tok,          # 2
        hidden_size=config.hidden_size,             # 4096
        intermediate_size=config.intermediate_size, # 14336
        quant_config=quant_config,
        prefix=f"{prefix}.block_sparse_moe",
        enable_eplb=enable_eplb,
    )

    # RMSNorm layers
    self.input_layernorm = RMSNorm(
        config.hidden_size,
        eps=config.rms_norm_eps
    )
    self.post_attention_layernorm = RMSNorm(
        config.hidden_size,
        eps=config.rms_norm_eps
    )

def forward(
    self,
    positions: torch.Tensor,
    hidden_states: torch.Tensor,
    residual: torch.Tensor | None,
) -> torch.Tensor:
    # Self Attention
    if residual is None:
        residual = hidden_states
        hidden_states = self.input_layernorm(hidden_states)
    else:
        hidden_states, residual = self.input_layernorm(
            hidden_states, residual
        )

    hidden_states = self.self_attn(
        positions=positions,
        hidden_states=hidden_states,
    )
```

```
        # MoE
        hidden_states, residual = self.post_attention_layernorm(
            hidden_states, residual
        )
        hidden_states = self.block_sparse_moe(hidden_states)

        return hidden_states, residual
```

Forward Pass 흐름:

```
Input: [batch_size, seq_len, hidden_size]
  |
  ├─▶ input_layernorm
  |
  ├─▶ self_attn (Multi-Head Attention)
  |     └─> [batch_size, seq_len, hidden_size]
  |
  ├─▶ Residual Connection
  |
  ├─▶ post_attention_layernorm
  |
  ├─▶ block_sparse_moe
  |     ├─> Router: [batch, seq, hidden] → [batch*seq, n_experts]
  |     ├─> Top-K selection
  |     ├─> Expert computation (parallelized)
  |     └─> Weighted sum
  |
  └─▶ Residual Connection

Output: [batch_size, seq_len, hidden_size]
```

2.3 Mixtral Config

```
# Mixtral-8x7B 기본 설정
MixtralConfig:
  hidden_size: 4096
  intermediate_size: 14336
  num_hidden_layers: 32
  num_attention_heads: 32
  num_key_value_heads: 8          # GQA

  # MoE specific
  num_local_experts: 8             # 총 Expert 수
```

```
  num_experts_per_tok: 2          # Top-K (활성화 Expert 수)

  # RoPE
  rope_theta: 1000000.0           # Long context (32K)
  max_position_embeddings: 32768

  # Normalization
  rms_norm_eps: 1e-5

  vocab_size: 32000
```

모델 크기 계산:

```
# Attention parameters (shared)
attention_params = (
    # Q, K, V projections
    hidden_size * (num_heads * head_dim) * 3 +
    # O projection
    (num_heads * head_dim) * hidden_size
)
= 4096 * (32 * 128) * 3 + (32 * 128) * 4096
= 4096 * 4096 * 3 + 4096 * 4096
= 67,108,864 params per layer

# MoE parameters
moe_params_per_expert = (
    # gate_proj + up_proj
    hidden_size * intermediate_size * 2 +
    # down_proj
    intermediate_size * hidden_size
)
= 4096 * 14336 * 2 + 14336 * 4096
= 176,160,768 params per expert

moe_total = moe_params_per_expert * num_experts
= 176,160,768 * 8
= 1,409,286,144 params per layer

# Router
router_params = hidden_size * num_experts
= 4096 * 8
= 32,768 params per layer

# Total per layer
total_per_layer = attention_params + moe_total + router_params
```

```
= 67,108,864 + 1,409,286,144 + 32,768
= 1,476,427,776 params

# Embedding + Output
embedding_params = vocab_size * hidden_size * 2
= 32000 * 4096 * 2
= 262,144,000 params

# Total model
total_params = total_per_layer * num_layers + embedding_params
= 1,476,427,776 * 32 + 262,144,000
= 47,267,832,832 params ≈ 47B
```

하지만 **Active Parameters** (실제 사용):

```
# Per token
active_attention = 67,108,864
active_experts = moe_params_per_expert * top_k
= 176,160,768 * 2
= 352,321,536

active_per_layer = active_attention + active_experts
= 67,108,864 + 352,321,536
= 419,430,400 params

active_total = active_per_layer * 32 + embedding_params
= 419,430,400 * 32 + 262,144,000
= 13,683,916,800 params ≈ 13.7B

→ Mixtral-8x7B의 active params: ~13.7B
→ 비슷한 성능의 dense model: Llama-70B (70B params)
→ 효율: 5x fewer active params!
```

---

3. MixtralMoE 구현 분석

이 섹션에서는 vLLM에서 Mixtral MoE를 어떻게 구현했는지 상세히 분석합니다.

3.1 MixtralMoE 클래스 전체 구조

```python
# vllm/model_executor/models/mixtral.py:75-154
class MixtralMoE(nn.Module):
    """Mixtral의 Sparse Mixture of Experts 레이어.

    구조:
    1. Router (Gate): 각 토큰을 어떤 Expert에 보낼지 결정
    2. Experts: 8개의 독립적인 FFN
    3. Top-K Selection: 각 토큰마다 2개의 Expert만 활성화
    """

    def __init__(
        self,
        num_experts: int,                    # 8 (Mixtral-8x7B)
        top_k: int,                          # 2
        hidden_size: int,                     # 4096
        intermediate_size: int,              # 14336
        params_dtype: torch.dtype | None = None,
        quant_config: QuantizationConfig | None = None,
        tp_size: int | None = None,
        prefix: str = "",
        enable_eplb: bool = False,
    ):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k
        self.enable_eplb = enable_eplb

        if params_dtype is None:
            params_dtype = torch.get_default_dtype()
        self.params_dtype = params_dtype

        # ═══════════════════════════════════════════════════════
        # 1. Router (Gate Network)
        # ═══════════════════════════════════════════════════════
        # Input:  [num_tokens, hidden_size]
        # Output: [num_tokens, num_experts]
        # 역할: 각 토큰이 어떤 Expert를 사용할지 점수 계산
        self.gate = ReplicatedLinear(
            hidden_size,
            num_experts,
            bias=False,  # Router는 bias 없음
            params_dtype=params_dtype,
            quant_config=None,
            prefix=f"{prefix}.gate",
        )
```

```
    # ============================================================
    # 2. Experts (Fused MoE)
    # ============================================================
    # 8개의 Expert를 효율적으로 관리
    # FusedMoE: Triton kernel로 최적화된 Expert 연산
    self.experts = FusedMoE(
        num_experts=num_experts,
        top_k=top_k,
        hidden_size=hidden_size,
        intermediate_size=intermediate_size,
        params_dtype=params_dtype,
        reduce_results=True,        # Expert 출력을 weighted sum으로 합침
        renormalize=True,           # Router weights를 renormalize
        quant_config=quant_config,
        tp_size=tp_size,
        prefix=f"{prefix}.experts",
    )

    # ============================================================
    # 3. Expert Parallelism Load Balancing (Optional)
    # ============================================================
    # Load imbalance 문제 해결을 위한 추가 기능
    if self.enable_eplb:
        self.eplb_gate = torch.nn.Linear(
            hidden_size,
            num_experts,
            bias=False,
            dtype=params_dtype,
        )

def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
    """
    Args:
        hidden_states: [num_tokens, hidden_size]

    Returns:
        output: [num_tokens, hidden_size]
    """
    # Step 1: Router가 각 토큰에 대한 Expert 점수 계산
    router_logits, _ = self.gate(hidden_states)
    # router_logits: [num_tokens, num_experts]

    # Step 2-4: FusedMoE에서 처리
    # - Top-K Expert 선택
    # - 선택된 Expert 실행
```

```
        # - Weighted sum으로 결과 합침
        final_hidden_states = self.experts(
            hidden_states=hidden_states,
            router_logits=router_logits,
        )

        # EPLB가 활성화된 경우 추가 처리
        if self.enable_eplb:
            eplb_logits = self.eplb_gate(hidden_states)
            # Load balancing logic...

        return final_hidden_states
```

핵심 포인트:

1. **Router (self.gate)**: `hidden_size → num_experts` 변환
2. **Experts (self.experts)**: FusedMoE로 8개 Expert를 효율적으로 관리
3. **Forward pass**: Router → Top-K → Expert computation → Weighted sum

3.2 Router (Gate Network) 상세 분석

Router의 역할

Router는 **각 토큰을 분석하여 어떤 Expert에게 보낼지 결정**하는 네트워크입니다.

```
# Router 구조
self.gate = ReplicatedLinear(
    in_features=4096,    # hidden_size
    out_features=8,      # num_experts
    bias=False,
)

# Parameters: 4096 × 8 = 32,768 (매우 작음!)
```

Forward Pass - Router

```
def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
    # Step 1: Router logits 계산
    router_logits, _ = self.gate(hidden_states)

    # 예시:
```

```
    # hidden_states: [4, 4096]  (4 tokens)
    # router_logits: [4, 8]     (각 토큰마다 8개 Expert 점수)
```

Router logits 예시 (실제 값):

```
# Token 0: "The"
router_logits[0] = [2.3, -0.5, 1.8, 0.2, -1.1, 3.0, 0.8, -0.3]
#                   E0   E1    E2   E3   E4    E5   E6   E7
#                                              ↑ Highest

# Token 1: "machine"
router_logits[1] = [1.2, 0.8, 2.5, -0.6, 1.9, 0.1, -0.4, 0.5]
#                   E0   E1   E2   E3    E4   E5   E6    E7
#                             ↑           ↑ Top-2

# Token 2: "learning"
router_logits[2] = [-0.3, 1.5, 0.9, 2.1, 0.4, -0.8, 1.7, 0.2]
#                    E0   E1   E2   E3   E4   E5    E6   E7
#                              ↑    ↑                ↑ High
```

Top-K Selection

Router logits에서 **Top-K개의 Expert를 선택**합니다.

```
# Mixtral: top_k = 2
# 각 토큰마다 상위 2개 Expert만 활성화

def select_top_k(router_logits: torch.Tensor, top_k: int = 2):
    """
    Args:
        router_logits: [num_tokens, num_experts] = [4, 8]
        top_k: 2

    Returns:
        topk_weights: [num_tokens, top_k] = [4, 2]
        topk_indices: [num_tokens, top_k] = [4, 2]
    """
    # Softmax 적용 (확률 분포로 변환)
    routing_weights = F.softmax(router_logits, dim=1)
    # routing_weights: [4, 8], 각 행의 합 = 1.0

    # Top-K 선택
    topk_weights, topk_indices = torch.topk(
        routing_weights,
```

```
        k=top_k,
        dim=1,
    )

    return topk_weights, topk_indices

# 예시 결과:
# Token 0: Top-2 = Expert 5, Expert 0
# topk_indices[0] = [5, 0]
# topk_weights[0] = [0.35, 0.28]  (Softmax 후)

# Token 1: Top-2 = Expert 2, Expert 4
# topk_indices[1] = [2, 4]
# topk_weights[1] = [0.31, 0.25]
```

Renormalization

Top-K weights를 **다시 정규화**하여 합이 1이 되도록 합니다.

```
# Before renormalization:
topk_weights[0] = [0.35, 0.28]  # Sum = 0.63 (< 1.0)

# After renormalization:
topk_weights[0] = [0.35/0.63, 0.28/0.63]
                = [0.556, 0.444]  # Sum = 1.0

# 이유: 선택된 Expert들만으로 100% 출력을 만들어야 함
```

Renormalization 코드:

```
if renormalize:
    # topk_weights의 합을 1로 만들기
    topk_weights = topk_weights / topk_weights.sum(dim=-1, keepdim=True)
```

3.3 Expert 구조

각 Expert는 **Llama의 MLP와 동일한 구조**를 가집니다.

Single Expert 구조

```
class Expert(nn.Module):
    """단일 Expert FFN (Llama MLP와 동일)"""
```

```python
    def __init__(self, hidden_size=4096, intermediate_size=14336):
        super().__init__()

        # Gate projection (SwiGLU의 gate)
        self.gate_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        # [4096, 14336]

        # Up projection (SwiGLU의 up)
        self.up_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
        # [4096, 14336]

        # Down projection (출력)
        self.down_proj = nn.Linear(intermediate_size, hidden_size, bias=False)
        # [14336, 4096]

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: [num_tokens, hidden_size]

        Returns:
            output: [num_tokens, hidden_size]
        """
        # SwiGLU activation
        gate = F.silu(self.gate_proj(x))  # [num_tokens, 14336]
        up = self.up_proj(x)               # [num_tokens, 14336]

        # Element-wise multiplication
        intermediate = gate * up           # [num_tokens, 14336]

        # Down projection
        output = self.down_proj(intermediate)  # [num_tokens, 4096]

        return output
```

Expert 파라미터 크기

```
# Single Expert
gate_proj_params = 4096 × 14336 = 58,720,256
up_proj_params   = 4096 × 14336 = 58,720,256
down_proj_params = 14336 × 4096 = 58,720,256
──────────────────────────────────────────────
Total per expert = 176,160,768 params ≈ 176M
```

```
# 8 Experts
Total = 176M × 8 = 1,409M params ≈ 1.4B
```

3.4 FusedMoE: Expert 병렬 처리

vLLM은 **FusedMoE**를 사용하여 여러 Expert를 효율적으로 실행합니다.

Naive Approach (비효율적)

```
# ❌ 비효율적인 방법: 각 Expert를 순차적으로 실행
def naive_moe_forward(hidden_states, topk_indices, topk_weights):
    outputs = []

    for i in range(num_tokens):
        token = hidden_states[i]  # [hidden_size]
        expert_ids = topk_indices[i]  # [top_k]
        weights = topk_weights[i]     # [top_k]

        token_output = 0
        for j in range(top_k):
            expert_id = expert_ids[j]
            weight = weights[j]

            # Expert 실행 (매우 느림!)
            expert_output = experts[expert_id](token)
            token_output += weight * expert_output

        outputs.append(token_output)

    return torch.stack(outputs)

# 문제:
# - 토큰마다 순차 처리 → 병렬화 불가
# - Expert 호출 오버헤드 큼
# - GPU 활용률 낮음
```

FusedMoE Approach (효율적)

```
# ✅ 효율적인 방법: 모든 토큰을 Expert별로 그룹화
def fused_moe_forward(hidden_states, router_logits):
    """
    핵심 아이디어:
```

```
    1. 토큰을 Expert별로 그룹화
    2. 각 Expert를 배치로 실행 (병렬화)
    3. 결과를 다시 토큰별로 재조합
    """

    # Step 1: Top-K 선택
    topk_weights, topk_indices = select_topk(router_logits)

    # Step 2: Expert별로 토큰 그룹화
    # expert_tokens[e] = Expert e를 사용하는 모든 토큰들
    expert_tokens = group_tokens_by_expert(topk_indices)

    # Step 3: 각 Expert를 배치로 실행
    expert_outputs = []
    for expert_id in range(num_experts):
        tokens_for_expert = expert_tokens[expert_id]

        if len(tokens_for_expert) > 0:
            # 배치로 한 번에 실행!
            output = experts[expert_id](tokens_for_expert)
            expert_outputs[expert_id] = output

    # Step 4: 결과 재조합
    final_output = combine_expert_outputs(
        expert_outputs,
        topk_indices,
        topk_weights,
    )

    return final_output
```

FusedMoE의 장점:

```
Naive:
  Token 0 → Expert 5 → wait
  Token 0 → Expert 0 → wait
  Token 1 → Expert 2 → wait
  Token 1 → Expert 4 → wait
  ...
  총 시간: O(num_tokens × top_k)

FusedMoE:
  [Token 0, Token 3] → Expert 0 (병렬!)
  [Token 1, Token 7] → Expert 2 (병렬!)
  [Token 2, Token 5] → Expert 4 (병렬!)
```

```
...
총 시간: O(num_experts) ≪ O(num_tokens × top_k)
```

3.5 Forward Pass 전체 흐름

전체 MoE forward pass를 단계별로 분석합니다.

```python
def mixtral_moe_forward_detailed(hidden_states: torch.Tensor):
    """
    Args:
        hidden_states: [num_tokens, hidden_size]
                     = [4, 4096] (예시: 4개 토큰)

    Returns:
        output: [num_tokens, hidden_size] = [4, 4096]
    """

    # ═══════════════════════════════════════════════════════════
    # Step 1: Router Computation
    # ═══════════════════════════════════════════════════════════
    router_logits = self.gate(hidden_states)
    # Input:  [4, 4096]
    # Output: [4, 8]  (4 tokens, 8 experts)

    # 예시 값:
    # router_logits = [
    #     [2.3, -0.5, 1.8,  0.2, -1.1,  3.0,  0.8, -0.3],  # Token 0
    #     [1.2,  0.8, 2.5, -0.6,  1.9,  0.1, -0.4,  0.5],  # Token 1
    #     [-0.3, 1.5, 0.9,  2.1,  0.4, -0.8,  1.7,  0.2],  # Token 2
    #     [0.6,  1.1, -0.2, 0.9,  2.3, -0.5,  1.8,  0.4],  # Token 3
    # ]

    # ═══════════════════════════════════════════════════════════
    # Step 2: Softmax Normalization
    # ═══════════════════════════════════════════════════════════
    routing_weights = F.softmax(router_logits, dim=1)
    # 각 행의 합 = 1.0
    # routing_weights = [
    #     [0.12, 0.07, 0.08, 0.15, 0.04, 0.24, 0.09, 0.21],  # Token 0
    #     [0.18, 0.12, 0.31, 0.09, 0.25, 0.11, 0.08, 0.16],  # Token 1
    #     [0.09, 0.21, 0.14, 0.25, 0.13, 0.06, 0.19, 0.13],  # Token 2
    #     [0.13, 0.19, 0.11, 0.16, 0.28, 0.08, 0.17, 0.14],  # Token 3
    # ]
```

```
# ═══════════════════════════════════════════════════════════
# Step 3: Top-K Selection
# ═══════════════════════════════════════════════════════════
topk_weights, topk_indices = torch.topk(routing_weights, k=2, dim=1)

# topk_indices (선택된 Expert IDs):
# [
#     [5, 7],  # Token 0 → Expert 5, 7
#     [2, 4],  # Token 1 → Expert 2, 4
#     [3, 1],  # Token 2 → Expert 3, 1
#     [4, 1],  # Token 3 → Expert 4, 1
# ]

# topk_weights (Softmax 후 weights):
# [
#     [0.24, 0.21],  # Token 0
#     [0.31, 0.25],  # Token 1
#     [0.25, 0.21],  # Token 2
#     [0.28, 0.19],  # Token 3
# ]

# ═══════════════════════════════════════════════════════════
# Step 4: Renormalization
# ═══════════════════════════════════════════════════════════
topk_weights = topk_weights / topk_weights.sum(dim=-1, keepdim=True)

# topk_weights (Renormalized):
# [
#     [0.533, 0.467],  # 0.24/(0.24+0.21), 0.21/(0.24+0.21)
#     [0.554, 0.446],  # 0.31/(0.31+0.25), 0.25/(0.31+0.25)
#     [0.543, 0.457],  # 0.25/(0.25+0.21), 0.21/(0.25+0.21)
#     [0.596, 0.404],  # 0.28/(0.28+0.19), 0.19/(0.28+0.19)
# ]

# ═══════════════════════════════════════════════════════════
# Step 5: Group Tokens by Expert
# ═══════════════════════════════════════════════════════════
# Expert 0: []
# Expert 1: [Token 2, Token 3]
# Expert 2: [Token 1]
# Expert 3: [Token 2]
# Expert 4: [Token 1, Token 3]
# Expert 5: [Token 0]
# Expert 6: []
# Expert 7: [Token 0]
```

```
# ═══════════════════════════════════════════════════════════
# Step 6: Execute Experts (Parallelized)
# ═══════════════════════════════════════════════════════════
# Expert 1 실행 (배치):
expert_1_input = hidden_states[[2, 3]]  # [2, 4096]
expert_1_output = experts[1](expert_1_input)  # [2, 4096]

# Expert 2 실행:
expert_2_input = hidden_states[[1]]  # [1, 4096]
expert_2_output = experts[2](expert_2_input)  # [1, 4096]

# Expert 4 실행 (배치):
expert_4_input = hidden_states[[1, 3]]  # [2, 4096]
expert_4_output = experts[4](expert_4_input)  # [2, 4096]

# ... (다른 experts도 동일)

# ═══════════════════════════════════════════════════════════
# Step 7: Weighted Sum (결과 조합)
# ═══════════════════════════════════════════════════════════
final_output = torch.zeros_like(hidden_states)  # [4, 4096]

# Token 0 output:
final_output[0] = (
    topk_weights[0, 0] * expert_5_output[0] +  # 0.533 × Expert 5
    topk_weights[0, 1] * expert_7_output[0]     # 0.467 × Expert 7
)

# Token 1 output:
final_output[1] = (
    topk_weights[1, 0] * expert_2_output[0] +  # 0.554 × Expert 2
    topk_weights[1, 1] * expert_4_output[0]     # 0.446 × Expert 4
)

# Token 2 output:
final_output[2] = (
    topk_weights[2, 0] * expert_3_output[0] +  # 0.543 × Expert 3
    topk_weights[2, 1] * expert_1_output[0]     # 0.457 × Expert 1
)

# Token 3 output:
final_output[3] = (
    topk_weights[3, 0] * expert_4_output[1] +  # 0.596 × Expert 4
    topk_weights[3, 1] * expert_1_output[1]     # 0.404 × Expert 1
)
```

```
    return final_output  # [4, 4096]
```

시각화:

```
Token 0 ──┬─▶ Router ─▶ [Expert 5: 0.533] ─┬─▶ Σ ─▶ Output 0
          └─▶ Router ─▶ [Expert 7: 0.467] ─┘

Token 1 ──┬─▶ Router ─▶ [Expert 2: 0.554] ─┬─▶ Σ ─▶ Output 1
          └─▶ Router ─▶ [Expert 4: 0.446] ─┘

Token 2 ──┬─▶ Router ─▶ [Expert 3: 0.543] ─┬─▶ Σ ─▶ Output 2
          └─▶ Router ─▶ [Expert 1: 0.457] ─┘

Token 3 ──┬─▶ Router ─▶ [Expert 4: 0.596] ─┬─▶ Σ ─▶ Output 3
          └─▶ Router ─▶ [Expert 1: 0.404] ─┘
```

3.6 코드 요약

전체 MixtralMoE forward pass를 간결하게 정리하면:

```
class MixtralMoE(nn.Module):
    def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
        # 1. Router: 각 토큰에 대한 Expert 점수 계산
        router_logits, _ = self.gate(hidden_states)

        # 2-7. FusedMoE에서 처리:
        #   - Top-K selection
        #   - Expert별 토큰 그룹화
        #   - Expert 병렬 실행
        #   - Weighted sum
        output = self.experts(hidden_states, router_logits)

        return output
```

핵심:

1. **Router**: 간단한 Linear layer로 Expert 선택
2. **FusedMoE**: Triton kernel로 최적화된 병렬 실행
3. **Weighted sum**: Renormalized weights로 결과 조합

---

4. FusedMoE 커널

이 섹션에서는 vLLM의 **FusedMoE** 구현을 상세히 분석합니다. FusedMoE는 Triton 커스텀 kernel을 사용하여 MoE 연산을 GPU에서 효율적으로 실행합니다.

4.1 FusedMoE 클래스 구조

```
# vllm/model_executor/layers/fused_moe/fused_moe.py:500-600
class FusedMoE(nn.Module):
    """Fused Mixture of Experts layer.

    핵심 기능:
    1. 여러 Expert를 하나의 큰 weight tensor로 통합
    2. Triton kernel로 Expert 연산 병렬 실행
    3. Top-K routing과 weighted sum을 GPU에서 직접 처리
    """

    def __init__(
        self,
        num_experts: int,                # 8
        top_k: int,                      # 2
        hidden_size: int,                # 4096
        intermediate_size: int,          # 14336
        params_dtype: torch.dtype | None = None,
        reduce_results: bool = True,     # Weighted sum 자동 처리
        renormalize: bool = True,        # Router weights renormalize
        use_grouped_topk: bool = False,  # Grouped top-k (Deepseek-V2)
        num_expert_group: int | None = None,
        topk_group: int | None = None,
        quant_config: QuantizationConfig | None = None,
        tp_size: int | None = None,
        prefix: str = "",
    ):
        super().__init__()

        # ═══════════════════════════════════════════════════════
        # Expert weights를 하나의 큰 tensor로 통합
        # ═══════════════════════════════════════════════════════
        # gate_proj: [num_experts, intermediate_size, hidden_size]
        #          = [8, 14336, 4096]
        self.gate_up_proj = FusedMoEParameter(
            num_experts=num_experts,
            hidden_size=hidden_size,
```

```
        intermediate_size=intermediate_size * 2,  # gate + up
        params_dtype=params_dtype,
        weight_loader=self.gate_up_weight_loader,
        prefix=f"{prefix}.gate_up_proj",
    )

    # down_proj: [num_experts, hidden_size, intermediate_size]
    #          = [8, 4096, 14336]
    self.down_proj = FusedMoEParameter(
        num_experts=num_experts,
        hidden_size=intermediate_size,
        intermediate_size=hidden_size,
        params_dtype=params_dtype,
        weight_loader=self.down_weight_loader,
        prefix=f"{prefix}.down_proj",
    )

    # Configuration
    self.num_experts = num_experts
    self.top_k = top_k
    self.reduce_results = reduce_results
    self.renormalize = renormalize
    self.use_grouped_topk = use_grouped_topk

def forward(
    self,
    hidden_states: torch.Tensor,
    router_logits: torch.Tensor,
) -> torch.Tensor:
    """
    Args:
        hidden_states: [num_tokens, hidden_size]
        router_logits: [num_tokens, num_experts]

    Returns:
        output: [num_tokens, hidden_size]
    """
    # Triton kernel 호출하여 MoE 연산 수행
    final_hidden_states = fused_moe(
        hidden_states=hidden_states,
        w1=self.gate_up_proj.weight,  # [8, 14336*2, 4096]
        w2=self.down_proj.weight,      # [8, 4096, 14336]
        router_logits=router_logits,   # [num_tokens, 8]
        top_k=self.top_k,              # 2
        renormalize=self.renormalize,  # True
        use_grouped_topk=self.use_grouped_topk,
```

```
        )

        return final_hidden_states
```

핵심 아이디어:

```
# ❌ Naive: 8개의 별도 Expert 모듈
experts = nn.ModuleList([
    ExpertFFN(...) for _ in range(8)
])
# → 각 Expert를 개별 호출 (느림)

# ✅ FusedMoE: 하나의 통합 weight tensor
gate_up_proj: [8, 28672, 4096]  # 8 experts × (gate + up)
down_proj:    [8, 4096, 14336]  # 8 experts × down
# → Triton kernel로 병렬 실행 (빠름!)
```

4.2 fused_moe() 함수

`fused_moe()` 함수는 실제 MoE 연산을 수행하는 진입점입니다.

```
# vllm/model_executor/layers/fused_moe/fused_moe.py:350-450
def fused_moe(
    hidden_states: torch.Tensor,      # [num_tokens, hidden_size]
    w1: torch.Tensor,                 # [num_experts, intermediate_size*2, hidde
    w2: torch.Tensor,                 # [num_experts, hidden_size, intermediate_
    router_logits: torch.Tensor,      # [num_tokens, num_experts]
    top_k: int,
    renormalize: bool = True,
    use_grouped_topk: bool = False,
    num_expert_group: int | None = None,
    topk_group: int | None = None,
) -> torch.Tensor:
    """
    MoE forward pass의 핵심 함수.

    단계:
    1. Top-K Expert 선택
    2. 토큰을 Expert별로 정렬
    3. Triton kernel로 Expert 연산 실행
    4. 결과를 원래 토큰 순서로 복원
    """

    # ═══════════════════════════════════════════════════
```

```
# Step 1: Top-K Expert Selection
# ============================================================
if use_grouped_topk:
    # Grouped top-k (Deepseek-V2용)
    topk_weights, topk_ids = grouped_topk(
        hidden_states=hidden_states,
        router_logits=router_logits,
        top_k=top_k,
        renormalize=renormalize,
        num_expert_group=num_expert_group,
        topk_group=topk_group,
    )
else:
    # Standard top-k (Mixtral용)
    topk_weights, topk_ids = topk_softmax(
        router_logits,
        top_k,
        renormalize=renormalize,
    )

# topk_weights: [num_tokens, top_k] - Renormalized weights
# topk_ids:     [num_tokens, top_k] - Selected expert IDs

# ============================================================
# Step 2: Token Sorting (Expert별 그룹화)
# ============================================================
# 토큰을 Expert ID별로 정렬하여 배치 처리 가능하게 함
sorted_token_ids, sorted_expert_ids, num_tokens_per_expert = (
    moe_align_block_size(
        topk_ids=topk_ids,
        block_size=64,  # GPU warp size 최적화
        num_experts=w1.shape[0],
    )
)

# sorted_token_ids:       토큰 인덱스 (Expert별로 정렬됨)
# sorted_expert_ids:      Expert ID (정렬됨)
# num_tokens_per_expert:  각 Expert가 처리할 토큰 수

# ============================================================
# Step 3: Triton Kernel 실행 (MoE Computation)
# ============================================================
# 핵심: Triton kernel로 모든 Expert 연산을 병렬 실행
intermediate_output = invoke_fused_moe_kernel(
    hidden_states=hidden_states,
    w1=w1,                          # Gate + Up weights
```

```
        w2=w2,                              # Down weights
        topk_weights=topk_weights,
        topk_ids=topk_ids,
        sorted_token_ids=sorted_token_ids,
        sorted_expert_ids=sorted_expert_ids,
        num_tokens_per_expert=num_tokens_per_expert,
    )

    # ═══════════════════════════════════════════════════════════
    # Step 4: 결과 재조합 및 복원
    # ═══════════════════════════════════════════════════════════
    # 정렬된 결과를 원래 토큰 순서로 복원
    final_output = finalize_moe_output(
        intermediate_output=intermediate_output,
        topk_weights=topk_weights,
        topk_ids=topk_ids,
    )

    return final_output  # [num_tokens, hidden_size]
```

4.3 topk_softmax - Top-K Selection

Top-K Expert를 선택하고 weights를 계산하는 함수입니다.

```
# vllm/model_executor/layers/fused_moe/fused_moe.py:100-150
def topk_softmax(
    router_logits: torch.Tensor,  # [num_tokens, num_experts]
    top_k: int,                   # 2
    renormalize: bool = True,
) -> tuple[torch.Tensor, torch.Tensor]:
    """
    Top-K expert selection with optional renormalization.

    Args:
        router_logits: [num_tokens, num_experts] = [4, 8]
        top_k: 2
        renormalize: True

    Returns:
        topk_weights: [num_tokens, top_k] = [4, 2]
        topk_ids:     [num_tokens, top_k] = [4, 2]
    """

    # ═══════════════════════════════════════════════════════════
```

```
    # Step 1: Softmax (확률 분포 변환)
    # ═══════════════════════════════════════════════════════════
    routing_weights = F.softmax(router_logits, dim=-1)
    # [num_tokens, num_experts]

    # 예시:
    # routing_weights[0] = [0.12, 0.07, 0.08, 0.15, 0.04, 0.24, 0.09, 0.21]
    #                        E0    E1    E2    E3    E4    E5    E6    E7
    #                                                        ↑           ↑  Top-2

    # ═══════════════════════════════════════════════════════════
    # Step 2: Top-K Selection
    # ═══════════════════════════════════════════════════════════
    topk_weights, topk_ids = torch.topk(
        routing_weights,
        k=top_k,
        dim=-1,
        sorted=True,  # 내림차순 정렬
    )

    # topk_weights[0] = [0.24, 0.21]  # Expert 5, 7의 weights
    # topk_ids[0]     = [5, 7]        # Expert IDs

    # ═══════════════════════════════════════════════════════════
    # Step 3: Renormalization (Optional)
    # ═══════════════════════════════════════════════════════════
    if renormalize:
        # Top-K weights의 합을 1로 만들기
        topk_weights = topk_weights / topk_weights.sum(dim=-1, keepdim=True)

        # Before: [0.24, 0.21]  (sum = 0.45)
        # After:  [0.533, 0.467] (sum = 1.0)

    return topk_weights, topk_ids
```

Renormalization 효과:

```
# Without renormalization:
Token 0 output = 0.24 × Expert5 + 0.21 × Expert7 = 0.45 × (expert outputs)
# → 출력 크기가 줄어듦 (sum < 1.0)

# With renormalization:
Token 0 output = 0.533 × Expert5 + 0.467 × Expert7 = 1.0 × (expert outputs)
# → 출력 크기 유지 (sum = 1.0)
```

4.4 moe_align_block_size - Token Sorting

토큰을 Expert별로 정렬하여 GPU에서 효율적으로 처리합니다.

```
# vllm/model_executor/layers/fused_moe/fused_moe.py:200-280
def moe_align_block_size(
    topk_ids: torch.Tensor,      # [num_tokens, top_k]
    block_size: int,             # 64 (GPU warp size)
    num_experts: int,            # 8
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """
    토큰을 Expert별로 정렬하고 block size에 맞춰 정렬.

    목적:
    1. Expert별로 토큰 그룹화 → 배치 처리
    2. Block size (64) 단위로 정렬 → GPU 효율성

    예시:
    Input topk_ids:
      Token 0 → [5, 7]
      Token 1 → [2, 4]
      Token 2 → [3, 1]
      Token 3 → [4, 1]

    Output:
      Expert 0: []
      Expert 1: [Token 2, Token 3]  ← 2 tokens
      Expert 2: [Token 1]           ← 1 token
      Expert 3: [Token 2]           ← 1 token
      Expert 4: [Token 1, Token 3]  ← 2 tokens
      Expert 5: [Token 0]           ← 1 token
      Expert 6: []
      Expert 7: [Token 0]           ← 1 token
    """

    num_tokens = topk_ids.shape[0]
    top_k = topk_ids.shape[1]

    # ═══════════════════════════════════════════════════════════
    # Step 1: Flatten top-k indices
    # ═══════════════════════════════════════════════════════════
    # topk_ids: [num_tokens, top_k] → [num_tokens * top_k]
    # 예: [4, 2] → [8]
    #
    # Before:
```

```
# [[5, 7],
#  [2, 4],
#  [3, 1],
#  [4, 1]]
#
# After:
# [5, 7, 2, 4, 3, 1, 4, 1]

flattened_topk_ids = topk_ids.flatten()  # [8]

# ═══════════════════════════════════════════════════════════
# Step 2: 각 Expert가 처리할 토큰 수 계산
# ═══════════════════════════════════════════════════════════
# Expert별로 몇 개의 토큰이 할당되었는지 계산
num_tokens_per_expert = torch.zeros(
    num_experts,
    dtype=torch.int32,
    device=topk_ids.device,
)

for expert_id in range(num_experts):
    num_tokens_per_expert[expert_id] = (
        flattened_topk_ids == expert_id
    ).sum()

# num_tokens_per_expert:
# [0, 2, 1, 1, 2, 1, 0, 1]
#  E0 E1 E2 E3 E4 E5 E6 E7

# ═══════════════════════════════════════════════════════════
# Step 3: Expert별로 토큰 정렬
# ═══════════════════════════════════════════════════════════
# flattened_topk_ids를 expert ID 순으로 정렬
sorted_expert_ids, sorted_indices = torch.sort(flattened_topk_ids)

# sorted_expert_ids:
# [1, 1, 2, 3, 4, 4, 5, 7]
#  E1 E1 E2 E3 E4 E4 E5 E7

# sorted_indices (원래 위치):
# [5, 7, 2, 4, 3, 6, 0, 1]
#  ↑  ↑  각 전문가에 할당된 토큰의 원래 인덱스

# ═══════════════════════════════════════════════════════════
# Step 4: Token ID 복원
# ═══════════════════════════════════════════════════════════
```

```
    # sorted_indices를 토큰 ID로 변환
    # sorted_indices: flattened index → token index
    sorted_token_ids = sorted_indices // top_k

    # sorted_indices = [5, 7, 2, 4, 3, 6, 0, 1]
    # sorted_token_ids = [2, 3, 1, 2, 1, 3, 0, 0]
    #                     ↑  ↑  Token 2, Token 3 → Expert 1

    # ═══════════════════════════════════════════════════════════
    # Step 5: Block alignment (GPU 최적화)
    # ═══════════════════════════════════════════════════════════
    # 각 Expert의 토큰 수를 block_size(64)의 배수로 올림
    # → GPU warp 효율성

    cumsum = num_tokens_per_expert.cumsum(dim=0)
    start_indices = torch.cat([
        torch.tensor([0], device=cumsum.device),
        cumsum[:-1]
    ])

    # Padding to block_size
    for expert_id in range(num_experts):
        start = start_indices[expert_id]
        count = num_tokens_per_expert[expert_id]

        # Block size로 올림
        padded_count = ((count + block_size - 1) // block_size) * block_size

        # Padding 추가 (필요시)
        if padded_count > count:
            num_tokens_per_expert[expert_id] = padded_count

    return sorted_token_ids, sorted_expert_ids, num_tokens_per_expert
```

Token Sorting 시각화:

```
Before sorting (by token):
  Token 0: [Expert 5, Expert 7]
  Token 1: [Expert 2, Expert 4]
  Token 2: [Expert 3, Expert 1]
  Token 3: [Expert 4, Expert 1]

After sorting (by expert):
  Expert 0: []
  Expert 1: [Token 2, Token 3]  ← Batch처리!
```

```
  Expert 2: [Token 1]
  Expert 3: [Token 2]
  Expert 4: [Token 1, Token 3]  ← Batch처리!
  Expert 5: [Token 0]
  Expert 6: []
  Expert 7: [Token 0]

→ Expert 1과 4는 2개 토큰을 배치로 처리 (효율적!)
```

4.5 Triton Kernel - invoke_fused_moe_kernel

실제 Expert 연산을 수행하는 Triton kernel입니다.

```python
# vllm/model_executor/layers/fused_moe/fused_moe.py:150-200
def invoke_fused_moe_kernel(
    hidden_states: torch.Tensor,          # [num_tokens, hidden_size]
    w1: torch.Tensor,                     # [num_experts, intermediate_size*2, h
    w2: torch.Tensor,                     # [num_experts, hidden_size, intermedi
    topk_weights: torch.Tensor,           # [num_tokens, top_k]
    topk_ids: torch.Tensor,               # [num_tokens, top_k]
    sorted_token_ids: torch.Tensor,       # Expert별 정렬된 토큰 IDs
    sorted_expert_ids: torch.Tensor,      # 정렬된 Expert IDs
    num_tokens_per_expert: torch.Tensor,  # 각 Expert의 토큰 수
) -> torch.Tensor:
    """
    Triton kernel을 호출하여 MoE 연산 실행.

    Kernel 동작:
    1. 각 Expert에 할당된 토큰들을 병렬로 처리
    2. SwiGLU activation 적용
    3. Weighted sum으로 결과 조합
    """

    num_tokens = hidden_states.shape[0]
    hidden_size = hidden_states.shape[1]
    intermediate_size = w1.shape[1] // 2  # gate + up → divide by 2

    # 출력 버퍼 할당
    output = torch.zeros(
        (num_tokens, hidden_size),
        dtype=hidden_states.dtype,
        device=hidden_states.device,
    )
```

```
    # ============================================================
    # Triton Kernel Configuration
    # ============================================================
    # Grid: 각 Expert마다 하나의 block
    grid = lambda meta: (num_experts,)

    # ============================================================
    # Triton Kernel 실행
    # ============================================================
    fused_moe_triton_kernel[grid](
        # Input tensors
        hidden_states_ptr=hidden_states,
        w1_ptr=w1,
        w2_ptr=w2,
        topk_weights_ptr=topk_weights,
        topk_ids_ptr=topk_ids,
        sorted_token_ids_ptr=sorted_token_ids,
        sorted_expert_ids_ptr=sorted_expert_ids,
        num_tokens_per_expert_ptr=num_tokens_per_expert,
        # Output
        output_ptr=output,
        # Dimensions
        num_tokens=num_tokens,
        hidden_size=hidden_size,
        intermediate_size=intermediate_size,
        num_experts=num_experts,
        top_k=top_k,
    )

    return output
```

4.6 Triton Kernel 상세 (Pseudo-code)

실제 Triton kernel의 핵심 로직을 pseudo-code로 설명합니다.

```
# Simplified Triton kernel (pseudo-code)
@triton.jit
def fused_moe_triton_kernel(
    hidden_states_ptr,      # Input tokens
    w1_ptr,                 # Gate + Up weights
    w2_ptr,                 # Down weights
    topk_weights_ptr,       # Router weights
    topk_ids_ptr,           # Expert IDs
```

```
    sorted_token_ids_ptr,   # Sorted token indices
    sorted_expert_ids_ptr,  # Sorted expert IDs
    num_tokens_per_expert_ptr,
    output_ptr,
    num_tokens,
    hidden_size,
    intermediate_size,
    num_experts,
    top_k,
):
    """
    각 Expert를 병렬로 실행하는 Triton kernel.

    Grid: (num_experts,)  → 각 Expert마다 하나의 block
    """

    # ============================================================
    # Step 1: Expert ID 결정
    # ============================================================
    expert_id = tl.program_id(0)  # Current expert ID (0-7)

    # 이 Expert가 처리할 토큰 수
    num_tokens_for_expert = tl.load(num_tokens_per_expert_ptr + expert_id)

    if num_tokens_for_expert == 0:
        return  # 이 Expert는 할당된 토큰이 없음

    # ============================================================
    # Step 2: Expert weights 로드
    # ============================================================
    # w1: [num_experts, intermediate_size*2, hidden_size]
    # w2: [num_experts, hidden_size, intermediate_size]

    w1_expert = w1_ptr + expert_id * intermediate_size * 2 * hidden_size
    w2_expert = w2_ptr + expert_id * hidden_size * intermediate_size

    # ============================================================
    # Step 3: 이 Expert에 할당된 각 토큰 처리
    # ============================================================
    for token_idx in range(num_tokens_for_expert):
        # 실제 토큰 ID 가져오기
        token_id = tl.load(sorted_token_ids_ptr + token_idx)

        # 토큰 hidden state 로드: [hidden_size]
        hidden_state = tl.load(hidden_states_ptr + token_id * hidden_size)
```

```
    # ═══════════════════════════════════════════════════════════
    # Step 3a: Gate & Up Projection (SwiGLU)
    # ═══════════════════════════════════════════════════════════
    # w1 @ hidden_state → [intermediate_size * 2]
    gate_up = matmul(w1_expert, hidden_state)

    # Split into gate and up
    gate = gate_up[:intermediate_size]      # [intermediate_size]
    up   = gate_up[intermediate_size:]      # [intermediate_size]

    # SwiGLU activation
    gate_activated = silu(gate)             # SiLU(gate)
    intermediate = gate_activated * up      # Element-wise mul

    # ═══════════════════════════════════════════════════════════
    # Step 3b: Down Projection
    # ═══════════════════════════════════════════════════════════
    # w2 @ intermediate → [hidden_size]
    expert_output = matmul(w2_expert, intermediate)

    # ═══════════════════════════════════════════════════════════
    # Step 3c: Weighted Sum (Router weight 적용)
    # ═══════════════════════════════════════════════════════════
    # 이 토큰의 top-k에서 현재 expert의 weight 찾기
    weight = find_weight_for_expert(
        topk_weights_ptr,
        topk_ids_ptr,
        token_id,
        expert_id,
        top_k,
    )

    # Weight 적용
    weighted_output = expert_output * weight

    # ═══════════════════════════════════════════════════════════
    # Step 3d: Atomic Add (여러 Expert 결과 합치기)
    # ═══════════════════════════════════════════════════════════
    # 한 토큰은 여러 Expert에서 처리되므로 atomic add 필요
    tl.atomic_add(
        output_ref + token_id * hidden_size,
        weighted_output,
    )

# ═══════════════════════════════════════════════════════════
```

```
    # 모든 Expert가 완료되면 output에 최종 결과 저장됨
    # ═══════════════════════════════════════════════════════════
```

Triton Kernel 실행 과정:

```
GPU Blocks (각 Expert마다):
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│  Block 0     │  │  Block 1     │  │  Block 4     │
│  Expert 0    │  │  Expert 1    │  │  Expert 4    │
│              │  │              │  │              │
│  No tokens   │  │ Token 2, 3   │  │ Token 1, 3   │
│              │  │              │  │              │
│  (Skip)      │  │ ┌──────────┐ │  │ ┌──────────┐ │
│              │  │ │w1 @ x    │ │  │ │w1 @ x    │ │
│              │  │ │SwiGLU    │ │  │ │SwiGLU    │ │
│              │  │ │w2 @ ...| │  | |w2 @ ...| |
│              │  │ └──────────┘ │  │ └──────────┘ │
└──────────────┘  └──────────────┘  └──────────────┘
                        |                  |
                        └──────────┬───────┘
                                   ▼
                            Atomic Add
                                   ▼
                        Final Output Buffer
```

4.7 FusedMoE의 장점

(1) 메모리 효율성

```python
# Naive approach: 8개의 별도 Expert 모듈
class NaiveExpert(nn.Module):
    def __init__(self):
        self.gate_proj = nn.Linear(4096, 14336)
        self.up_proj   = nn.Linear(4096, 14336)
        self.down_proj = nn.Linear(14336, 4096)

experts = [NaiveExpert() for _ in range(8)]

# 문제:
# - 8개의 독립적인 모듈 → 메모리 단편화
# - Parameter 접근이 비효율적 (cache miss)
# - 각 forward call마다 오버헤드
```

```
# FusedMoE: 통합 weight tensor
class FusedMoE(nn.Module):
    def __init__(self):
        # All experts in one tensor!
        self.gate_up = nn.Parameter(torch.randn(8, 28672, 4096))
        self.down    = nn.Parameter(torch.randn(8, 4096, 14336))

# 장점:
# - 연속된 메모리 → cache friendly
# - Single kernel call로 모든 Expert 실행
# - Memory coalescing 최적화
```

(2) 계산 효율성

```
# Naive: O(num_tokens × top_k) kernel launches
for token in tokens:
    for expert_id in token.top_k_experts:
        output[token] += experts[expert_id](token) * weight

# Time: 100 tokens × 2 experts = 200 kernel launches

# FusedMoE: O(num_experts) kernel launches
for expert_id in range(num_experts):
    tokens_for_expert = group_by_expert(expert_id)
    output[tokens_for_expert] = expert[expert_id](tokens_for_expert)

# Time: 8 expert blocks (병렬 실행!)
# → 25x speedup!
```

(3) Batch Processing

```
# Token grouping으로 배치 처리:
Expert 1: [Token 2, Token 3]     ← 2개를 배치로!
Expert 4: [Token 1, Token 3, ...]← N개를 배치로!

# GPU matrix multiplication efficiency:
# - Small batch (1 token):  ~10% GPU utilization
# - Large batch (64 tokens): ~80% GPU utilization

# → Batch size가 클수록 효율적!
```

4.8 FusedMoE 요약

핵심 아이디어:

1. **Weight Fusion**: 모든 Expert를 하나의 큰 tensor로 통합
2. **Token Sorting**: Expert별로 토큰 그룹화 → 배치 처리
3. **Triton Kernel**: GPU에서 모든 Expert를 병렬 실행
4. **Atomic Add**: 여러 Expert 결과를 효율적으로 합침

성능 향상:

```
Naive MoE:     200 kernel launches
FusedMoE:      8 kernel launches
──────────────────────────────────
Speedup:       25x faster!

Memory access: Scattered → Coalesced
GPU util:      10% → 80%
Throughput:    100 tokens/s → 2500 tokens/s
```

---

5. Deepseek-V2 MoE

Deepseek-V2는 Mixtral과 다른 MoE 구조를 사용합니다. **Shared Experts**와 **Grouped Top-K Routing**이 핵심 차이점입니다.

5.1 Deepseek-V2 MoE 개요

Deepseek-V2 MoE의 특징:

```
┌──────────────────────────────────────────────────────────┐
│                   Deepseek-V2 MoE Layer                  │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Input (hidden_states)                                   │
│    │                                                     │
│    ├──────────────────┬──────────────────┐               │
│    │                  │                  │               │
│    ▼                  ▼                  ▼               │
│  Shared            Router           Routed               │
│  Experts          (Grouped)         Experts              │
│  (항상 활성)       Top-K            (Top-K만 활성)        │
```

```
│    │                  │                  │              │
│    │                  │                  │              │
│    ▼                  ▼                  ▼              │
│  Always           Select K            Sparse            │
│  Active           from Groups         Activation        │
│    │                  │                  │              │
│    └──────────────────┴──────────────────┘              │
│                       │                                  │
│                       ▼                                  │
│                  Output (weighted sum)                   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

핵심 차이점:

특징	Mixtral	Deepseek-V2
Shared Experts	없음	있음 (항상 활성)
Routed Experts	8개	64개 (더 많음)
Top-K	2개	6개 (더 많음)
Routing	Standard Top-K	Grouped Top-K
Expert Groups	없음	8 groups
Total params	47B	236B
Active params	13.7B	~21B

5.2 Shared Experts

Deepseek-V2의 핵심 혁신: **모든 토큰이 항상 사용하는 Shared Experts**

Shared Experts 구조

```
# vllm/model_executor/models/deepseek_v2.py:218-226
if config.n_shared_experts is None:
    self.shared_experts = None
else:
    # Shared experts: 모든 토큰이 사용하는 experts
    intermediate_size = config.moe_intermediate_size * config.n_shared_experts
```

```
    self.shared_experts = DeepseekV2MLP(
        hidden_size=config.hidden_size,
        intermediate_size=intermediate_size,  # n_shared_experts배!
        hidden_act=config.hidden_act,
        quant_config=quant_config,
        reduce_results=False,
        prefix=f"{prefix}.shared_experts",
    )

# Deepseek-V2: n_shared_experts = 2
# → intermediate_size = 1536 × 2 = 3072
```

Shared Experts vs Routed Experts:

```
# Shared Experts (항상 활성):
shared_output = shared_experts(hidden_states)  # 모든 토큰

# Routed Experts (Top-K만 활성):
routed_output = select_and_run_experts(hidden_states, top_k=6)  # 일부만

# Final output:
output = shared_output + routed_scaling_factor × routed_output
```

장점:

1. **안정성**: 모든 토큰이 기본 처리를 받음 (shared experts)
2. **전문화**: 특정 토큰만 추가 전문 처리 (routed experts)
3. **Load balancing**: Shared experts가 기본 부하 처리

비유:

```
Shared Experts   = 모든 학생이 듣는 필수 수업 (수학, 국어)
Routed Experts   = 일부 학생만 듣는 선택 수업 (물리, 화학, 생물)

→ 모든 학생은 기본 교육 + 자신에게 맞는 전문 교육
```

5.3 Grouped Top-K Routing

Deepseek-V2의 두 번째 혁신: **Expert를 그룹으로 나눠서 routing**

Standard Top-K (Mixtral)

```
# Mixtral: 8개 experts 중 2개 선택
Experts: [E0, E1, E2, E3, E4, E5, E6, E7]
         └──────────── 전체 ────────────┘
                    ↓
            Top-K=2 선택
                    ↓
         Selected: [E3, E7]

# 문제:
# - Expert 수가 많아지면 (64개) routing 어려움
# - 모든 expert를 공정하게 비교하기 힘듦
```

Grouped Top-K (Deepseek-V2)

```
# Deepseek-V2: 64 experts, 8 groups, top_k=6

# Step 1: Expert를 8개 그룹으로 나눔
Group 0: [E0,  E1,  E2,  E3,  E4,  E5,  E6,  E7]   ← 8 experts
Group 1: [E8,  E9,  E10, E11, E12, E13, E14, E15]
Group 2: [E16, E17, E18, E19, E20, E21, E22, E23]
Group 3: [E24, E25, E26, E27, E28, E29, E30, E31]
Group 4: [E32, E33, E34, E35, E36, E37, E38, E39]
Group 5: [E40, E41, E42, E43, E44, E45, E46, E47]
Group 6: [E48, E49, E50, E51, E52, E53, E54, E55]
Group 7: [E56, E57, E58, E59, E60, E61, E62, E63]

# Step 2: 각 그룹에서 topk_group개 선택
topk_group = 1  # 각 그룹에서 1개

Group 0 → E3   (점수 가장 높음)
Group 1 → E12
Group 2 → E19
Group 3 → E27
Group 4 → E35
Group 5 → E42
Group 6 → E51
Group 7 → E60

# 총 8개 선택됨 (각 그룹에서 1개씩)

# Step 3: 8개 중에서 최종 top_k=6개 선택
Candidates: [E3, E12, E19, E27, E35, E42, E51, E60]
                    ↓ Top-6 선택
Final selected: [E12, E19, E27, E35, E42, E51]
```

Grouped Top-K의 장점:

1. **Expert 다양성**: 각 그룹에서 1개씩 선택 → 다양한 그룹의 experts 활용
2. **Routing 안정성**: 한 그룹에서 모든 expert가 선택되는 일 방지
3. **Load Balancing**: 그룹 단위로 부하 분산

코드 예시:

```
# vllm/model_executor/models/deepseek_v2.py:228-255
self.experts = SharedFusedMoE(
    shared_experts=self.shared_experts,
    gate=self.gate,
    num_experts=config.n_routed_experts,  # 64
    top_k=config.num_experts_per_tok,     # 6
    use_grouped_topk=True,                # Grouped routing!
    num_expert_group=config.n_group,      # 8 groups
    topk_group=config.topk_group,         # 1 per group
    ...
)

# Config:
# - n_routed_experts = 64
# - num_experts_per_tok = 6
# - n_group = 8
# - topk_group = 1

# Routing:
# 1. 64 experts → 8 groups (8 experts per group)
# 2. 각 그룹에서 1개 선택 → 8개
# 3. 8개 중 최종 6개 선택
```

5.4 DeepseekV2MoE 클래스 구조

```
# vllm/model_executor/models/deepseek_v2.py:154-256
class DeepseekV2MoE(nn.Module):
    """Deepseek-V2 MoE with Shared Experts and Grouped Top-K."""

    def __init__(
        self,
        config: DeepseekV2Config,
        parallel_config: ParallelConfig,
        quant_config: QuantizationConfig | None = None,
        prefix: str = "",
```

```
):
    super().__init__()

    # ═══════════════════════════════════════════════════════════
    # 1. Configuration
    # ═══════════════════════════════════════════════════════════
    self.n_routed_experts = config.n_routed_experts    # 64
    self.n_shared_experts = config.n_shared_experts    # 2

    # Routed scaling factor (Deepseek-V2 특유)
    self.routed_scaling_factor = config.routed_scaling_factor
    # → 0.125 (1/8)

    # ═══════════════════════════════════════════════════════════
    # 2. Router (Gate Network)
    # ═══════════════════════════════════════════════════════════
    self.gate = ReplicatedLinear(
        config.hidden_size,              # 5120
        config.n_routed_experts,         # 64
        bias=False,
        prefix=f"{prefix}.gate",
    )

    # ═══════════════════════════════════════════════════════════
    # 3. Shared Experts (항상 활성)
    # ═══════════════════════════════════════════════════════════
    if config.n_shared_experts is not None:
        # Shared experts: 모든 토큰이 사용
        intermediate_size = (
            config.moe_intermediate_size * config.n_shared_experts
        )
        # = 1536 × 2 = 3072

        self.shared_experts = DeepseekV2MLP(
            hidden_size=config.hidden_size,
            intermediate_size=intermediate_size,
            hidden_act=config.hidden_act,  # silu
            quant_config=quant_config,
            reduce_results=False,
            prefix=f"{prefix}.shared_experts",
        )
    else:
        self.shared_experts = None

    # ═══════════════════════════════════════════════════════════
    # 4. Routed Experts (Sparse MoE)
```

```
    # ═══════════════════════════════════════════════════════════
    self.experts = SharedFusedMoE(
        shared_experts=self.shared_experts,
        gate=self.gate,
        num_experts=config.n_routed_experts,      # 64
        top_k=config.num_experts_per_tok,         # 6
        hidden_size=config.hidden_size,            # 5120
        intermediate_size=config.moe_intermediate_size,  # 1536
        reduce_results=False,
        renormalize=config.norm_topk_prob,        # True
        # Grouped Top-K!
        use_grouped_topk=True,
        num_expert_group=config.n_group,          # 8
        topk_group=config.topk_group,             # 1
        routed_scaling_factor=self.routed_scaling_factor,
        prefix=f"{prefix}.experts",
    )

def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
    """
    Args:
        hidden_states: [num_tokens, hidden_size]

    Returns:
        output: [num_tokens, hidden_size]
    """

    # ═══════════════════════════════════════════════════════════
    # Step 1: Router Logits 계산
    # ═══════════════════════════════════════════════════════════
    router_logits, _ = self.gate(hidden_states)
    # [num_tokens, n_routed_experts] = [4, 64]

    # ═══════════════════════════════════════════════════════════
    # Step 2: SharedFusedMoE 실행
    # ═══════════════════════════════════════════════════════════
    # → Shared experts + Routed experts 동시 실행
    shared_output, routed_output = self.experts(
        hidden_states=hidden_states,
        router_logits=router_logits,
    )

    # shared_output:  [num_tokens, hidden_size]  (항상 있음)
    # routed_output:  [num_tokens, hidden_size]  (Top-K만)

    # ═══════════════════════════════════════════════════════════
```

```
    # Step 3: Scaling and Combination
    # ═══════════════════════════════════════════════════════════
    # Routed output에 scaling factor 적용
    routed_output *= self.routed_scaling_factor  # × 0.125

    # Final output: shared + scaled routed
    final_output = shared_output + routed_output

    return final_output
```

5.5 Forward Pass 상세 분석

Deepseek-V2 MoE의 전체 forward pass를 분석합니다.

```
def deepseek_v2_moe_forward_detailed(hidden_states: torch.Tensor):
    """
    Args:
        hidden_states: [num_tokens, hidden_size] = [4, 5120]

    Returns:
        output: [num_tokens, hidden_size] = [4, 5120]
    """

    # ═══════════════════════════════════════════════════════════
    # Part A: Shared Experts (항상 실행)
    # ═══════════════════════════════════════════════════════════
    # 모든 토큰이 shared experts를 통과
    shared_output = shared_experts(hidden_states)
    # Input:  [4, 5120]
    # Output: [4, 5120]

    # Shared experts는 더 큰 intermediate size 사용:
    # intermediate_size = 1536 × 2 = 3072
    # (일반 expert는 1536)

    # ═══════════════════════════════════════════════════════════
    # Part B: Router Computation
    # ═══════════════════════════════════════════════════════════
    router_logits = gate(hidden_states)
    # Input:  [4, 5120]
    # Output: [4, 64]  (64 routed experts)

    # 예시:
    # router_logits[0] = [0.1, 0.3, ..., 0.8, ...]  (64개 값)
```

```
# ═══════════════════════════════════════════════════════════
# Part C: Grouped Top-K Selection
# ═══════════════════════════════════════════════════════════
# Step C1: Softmax
routing_weights = F.softmax(router_logits, dim=-1)
# [4, 64]

# Step C2: Reshape to groups
# 64 experts → 8 groups × 8 experts
routing_weights_groups = routing_weights.reshape(4, 8, 8)
# [num_tokens, num_groups, experts_per_group]
# [4, 8, 8]

# Step C3: 각 그룹에서 topk_group개 선택
group_topk_weights, group_topk_indices = torch.topk(
    routing_weights_groups,
    k=topk_group,  # 1
    dim=-1,
)
# group_topk_weights:  [4, 8, 1]
# group_topk_indices:  [4, 8, 1]

# Token 0, Group 0에서 선택된 expert:
# group_topk_indices[0, 0, 0] = 3  (E3)

# Step C4: Flatten back
group_topk_weights = group_topk_weights.flatten(1, 2)   # [4, 8]
group_topk_indices = group_topk_indices.flatten(1, 2)   # [4, 8]

# Step C5: 8개 중 최종 top_k=6개 선택
final_topk_weights, final_topk_indices = torch.topk(
    group_topk_weights,
    k=top_k,  # 6
    dim=-1,
)
# final_topk_weights:  [4, 6]
# final_topk_indices:  [4, 6]

# Token 0 선택 결과 예시:
# final_topk_indices[0] = [12, 19, 27, 35, 42, 51]
# → Group 1, 2, 3, 4, 5, 6에서 각각 1개

# Step C6: Renormalization
final_topk_weights = (
    final_topk_weights / final_topk_weights.sum(dim=-1, keepdim=True)
```

```
    )

    # ═══════════════════════════════════════════════════════════
    # Part D: Expert Execution (FusedMoE)
    # ═══════════════════════════════════════════════════════════
    # Mixtral과 동일한 FusedMoE 메커니즘 사용
    routed_output = fused_moe(
        hidden_states=hidden_states,
        w1=experts_w1,  # [64, 3072, 5120]
        w2=experts_w2,  # [64, 5120, 1536]
        topk_weights=final_topk_weights,
        topk_ids=final_topk_indices,
    )
    # Output: [4, 5120]

    # ═══════════════════════════════════════════════════════════
    # Part E: Scaling and Combination
    # ═══════════════════════════════════════════════════════════
    # Routed output scaling
    routed_output *= routed_scaling_factor  # × 0.125

    # Final combination
    final_output = shared_output + routed_output

    # 예시 (Token 0):
    # shared_output[0]  = [1.2, 0.8, -0.5, ...]
    # routed_output[0]  = [0.3, 0.1,  0.2, ...] (scaled)
    # final_output[0]   = [1.5, 0.9, -0.3, ...]

    return final_output  # [4, 5120]
```

시각화:

```
Token 0 ──┬─▶ Shared Experts ──────────────┬─▶ Sum ─▶ Output
          │   (항상 실행)                  │
          │                                 │
          └─▶ Router ─▶ Grouped Top-K      │
               │                           │
               ├─▶ Group 0: E3    ─┐        │
               ├─▶ Group 1: E12   ─┤        │
               ├─▶ Group 2: E19   ─┤        │
               ├─▶ Group 3: E27   ─┼─▶ Top-6 ─▶ × 0.125 ─┘
               ├─▶ Group 4: E35   ─┤
               ├─▶ Group 5: E42   ─┤
```

```
                ├─▶ Group 6: E51    ┤
                └─▶ Group 7: E60    ┘
```

5.6 Routed Scaling Factor

Deepseek-V2는 **routed output에 scaling factor를 적용**합니다.

```
# routed_scaling_factor = 0.125 (1/8)

final_output = shared_output + routed_scaling_factor × routed_output
             = shared_output + 0.125 × routed_output
```

이유:

1. **안정성**: Routed experts의 영향을 줄여서 학습 안정화
2. **균형**: Shared experts가 주된 역할, routed experts는 보조
3. **FP16 overflow 방지**: 출력 크기 제한

효과:

```
# Without scaling:
shared_output  = [1.0, 0.5, -0.3]
routed_output  = [2.0, 1.5,  0.8]  (큰 값!)
final          = [3.0, 2.0,  0.5]  (overflow 위험)

# With scaling (0.125):
shared_output  = [1.0, 0.5, -0.3]
routed_output  = [0.25, 0.19, 0.1]  (작아짐)
final          = [1.25, 0.69, -0.2] (안정적)
```

5.7 Deepseek-V2 Config

```
# Deepseek-V2-236B 기본 설정
DeepseekV2Config:
  hidden_size: 5120
  intermediate_size: 12288              # Dense layers
  moe_intermediate_size: 1536           # MoE layers (작음!)

  num_hidden_layers: 60
  num_attention_heads: 128
  num_key_value_heads: 128
```

```
  # MoE specific
  n_shared_experts: 2                    # Shared experts
  n_routed_experts: 64                   # Routed experts
  num_experts_per_tok: 6                 # Top-K

  # Grouped Top-K
  n_group: 8                             # Number of groups
  topk_group: 1                          # Select 1 per group

  # Scaling
  routed_scaling_factor: 0.125           # 1/8

  vocab_size: 102400
```

파라미터 계산:

```
# Shared Experts (per layer)
shared_intermediate = moe_intermediate_size × n_shared_experts
                    = 1536 × 2 = 3072

shared_params = (
    hidden_size × shared_intermediate × 2 +  # gate + up
    shared_intermediate × hidden_size         # down
)
= 5120 × 3072 × 2 + 3072 × 5120
= 47,185,920 params

# Routed Experts (per layer)
routed_params_per_expert = (
    hidden_size × moe_intermediate_size × 2 +
    moe_intermediate_size × hidden_size
)
= 5120 × 1536 × 2 + 1536 × 5120
= 23,592,960 params per expert

routed_total = routed_params_per_expert × n_routed_experts
             = 23,592,960 × 64
             = 1,509,949,440 params

# Total MoE params (per layer)
moe_layer_params = shared_params + routed_total
                 = 47,185,920 + 1,509,949,440
                 = 1,557,135,360 params ≈ 1.56B per layer
```

```
# Active params (per token)
active_shared = shared_params
              = 47,185,920

active_routed = routed_params_per_expert × num_experts_per_tok
              = 23,592,960 × 6
              = 141,557,760

active_per_layer = active_shared + active_routed
                 = 47,185,920 + 141,557,760
                 = 188,743,680 params ≈ 189M

# Total model
# 60 layers × 1.56B + attention + embedding
# ≈ 236B total params
# ≈ 21B active params per token
```

---

6. Mixtral vs Deepseek-V2 비교

이 섹션에서는 Mixtral과 Deepseek-V2의 MoE 구조를 비교 분석합니다.

6.1 핵심 차이점 요약

특징	Mixtral-8x7B	Deepseek-V2-236B
모델 크기	47B params	236B params
Active params	13.7B	21B
Efficiency	5x (vs 70B dense)	11x (vs 236B dense)
Shared Experts	❌ 없음	✅ 2개 (항상 활성)
Routed Experts	8개	64개
Top-K	2	6
Routing	Standard Top-K	Grouped Top-K (8 groups)
Expert Groups	없음	8 groups × 8 experts

특징	Mixtral-8x7B	Deepseek-V2-236B
Scaling Factor	없음	0.125 (routed output)
Hidden Size	4096	5120
MoE Intermediate	14336	1536 (작음!)
Shared Intermediate	-	3072 (2 experts)

6.2 아키텍처 비교

Mixtral 아키텍처

```
Input (hidden_states)
  |
  ├─▶ Router (Gate)
  |     |
  |     ├─▶ Softmax → Top-2 Selection
  |     |
  |     └─▶ Expert Selection:
  |           ├─ Expert 0  ─┐
  |           ├─ Expert 1   |
  |           ├─ Expert 2   ├─ 2개 선택
  |           ├─ Expert 3   |
  |           ├─ Expert 4   |
  |           ├─ Expert 5   |
  |           ├─ Expert 6   |
  |           └─ Expert 7  ─┘
  |                |
  |                └─▶ Weighted Sum
  |
  └─▶ Output

특징:
- 단순한 Top-K routing
- 모든 Expert가 동일한 intermediate size
- Scaling 없음
```

Deepseek-V2 아키텍처

```
Input (hidden_states)
  |
  ├──────────────┬──────────────────┐
  |              |                  |
  ▼              ▼                  ▼
Shared        Router          Routed Experts
Experts       (Gate)          (64개)
(2개)           |
  |             ├─▶ Softmax
Always          |
Active          ├─▶ Grouped Top-K:
  |             |     ├─ Group 0 (E0-E7)   → 1개
  |             |     ├─ Group 1 (E8-E15)  → 1개
  |             |     ├─ Group 2 (E16-E23) → 1개
  |             |     ├─ Group 3 (E24-E31) → 1개
  |             |     ├─ Group 4 (E32-E39) → 1개
  |             |     ├─ Group 5 (E40-E47) → 1개
  |             |     ├─ Group 6 (E48-E55) → 1개
  |             |     └─ Group 7 (E56-E63) → 1개
  |             |           |
  |             |           └─▶ 8개 → Top-6 Selection
  |             |                    |
  |             └────────────────────┼─▶ × 0.125 (Scaling)
  |                                  |
  └──────────────────────────────────┴─▶ Sum → Output
```

```
특징:
- Shared experts 항상 실행
- Grouped routing (expert 다양성)
- Routed output scaling (안정성)
```

6.3 Routing 메커니즘 비교

Mixtral: Standard Top-K

```
# Step 1: Router logits
router_logits = gate(hidden_states)  # [tokens, 8]

# Step 2: Softmax
weights = F.softmax(router_logits, dim=-1)

# Step 3: Top-2 selection
topk_weights, topk_ids = torch.topk(weights, k=2, dim=-1)
```

```
# Step 4: Renormalize
topk_weights = topk_weights / topk_weights.sum(dim=-1, keepdim=True)

# 예시:
# Token 0 → Expert 5 (0.65), Expert 0 (0.35)
# Token 1 → Expert 2 (0.58), Expert 4 (0.42)

# 장점:
# - 단순하고 빠름
# - 구현 용이

# 단점:
# - Expert 수가 많으면 routing 어려움
# - Load imbalance 발생 가능
```

Deepseek-V2: Grouped Top-K

```
# Step 1: Router logits
router_logits = gate(hidden_states)  # [tokens, 64]

# Step 2: Softmax
weights = F.softmax(router_logits, dim=-1)

# Step 3: Reshape to groups
weights_grouped = weights.reshape(tokens, 8, 8)  # 8 groups

# Step 4: Select 1 from each group
group_topk_weights, group_topk_ids = torch.topk(
    weights_grouped,
    k=1,  # topk_group
    dim=-1,
)

# Step 5: Flatten
candidates_weights = group_topk_weights.flatten(1, 2)  # [tokens, 8]
candidates_ids = group_topk_ids.flatten(1, 2)

# Step 6: Final Top-6 from 8 candidates
final_topk_weights, final_topk_ids = torch.topk(
    candidates_weights,
    k=6,
    dim=-1,
)

# Step 7: Renormalize
```

```
final_topk_weights = final_topk_weights / final_topk_weights.sum(dim=-1, keepdim

# 예시:
# Token 0 → E12 (0.18), E19 (0.17), E27 (0.16), E35 (0.16), E42 (0.17), E16 (0.1
#         → 다양한 그룹에서 선택!

# 장점:
# - Expert 다양성 보장 (각 그룹에서 선택)
# - Load balancing 개선
# - 64개 expert도 효율적으로 관리

# 단점:
# - 더 복잡한 구현
# - 약간의 오버헤드
```

6.4 파라미터 효율성 비교

Mixtral-8x7B

```
# Single Expert
expert_params = (
    4096 × 14336 × 2 +  # gate + up
    14336 × 4096         # down
) = 176M per expert

# Total (8 experts)
total_moe_params = 176M × 8 = 1.4B per layer

# Active (2 experts)
active_moe_params = 176M × 2 = 352M per layer

# Efficiency vs Dense
Mixtral-8x7B (13.7B active) ≈ Llama-70B performance
→ 5x parameter efficiency
```

Deepseek-V2-236B

```
# Shared Experts (항상 활성)
shared_params = (
    5120 × 3072 × 2 +  # gate + up (2 experts)
    3072 × 5120         # down
) = 47M
```

```
# Routed Expert (작음!)
routed_expert_params = (
    5120 × 1536 × 2 +  # gate + up
    1536 × 5120         # down
) = 24M per expert

# Total (2 shared + 64 routed)
total_moe_params = 47M + (24M × 64) = 1.58B per layer

# Active (2 shared + 6 routed)
active_moe_params = 47M + (24M × 6) = 191M per layer

# Efficiency vs Dense
Deepseek-V2 (21B active) ≈ 236B dense model performance
→ 11x parameter efficiency
```

비교:

```
Model          Total    Active   Efficiency
──────────────────────────────────────────
Mixtral        47B      13.7B    5x
Deepseek-V2    236B     21B      11x

Deepseek-V2가 더 효율적!
→ Shared experts + 더 많은 experts
→ 작은 intermediate size per expert
```

6.5 장단점 비교

Mixtral

장점:

1. **단순성**: 구현이 간단하고 이해하기 쉬움
2. **빠른 routing**: Standard Top-K로 빠른 expert 선택
3. **적은 오버헤드**: Routing 오버헤드 최소화
4. **검증된 구조**: Mistral AI의 검증된 아키텍처

단점:

1. **Scalability**: Expert 수를 늘리기 어려움 (8개 → 16개도 힘듦)
2. **Load imbalance**: 특정 expert에 부하 집중 가능
3. **전문화 제한**: 8개 expert로는 세밀한 전문화 어려움

4. **No fallback**: Shared experts 없어서 기본 처리 부족

Deepseek-V2

장점:

1. **Scalability**: 64개 expert도 효율적으로 관리
2. **안정성**: Shared experts가 기본 처리 보장
3. **Expert 다양성**: Grouped routing으로 다양한 expert 활용
4. **높은 효율**: 11x parameter efficiency (vs Mixtral 5x)
5. **Load balancing**: 그룹 단위 routing으로 부하 분산

단점:

1. **복잡성**: 구현이 더 복잡함
2. **Routing 오버헤드**: Grouped Top-K가 약간 느림
3. **메모리**: 64 experts + shared experts = 더 많은 메모리
4. **튜닝 필요**: Scaling factor 등 추가 하이퍼파라미터

6.6 Use Case 비교

Mixtral이 적합한 경우

```
1. 작은 모델 (< 100B params)
2. 빠른 inference가 중요
3. 단순한 구조 선호
4. Expert 수가 적어도 됨 (8-16개)

예시:
- Mistral-7B → Mixtral-8x7B (47B)
- 개인/소규모 서비스
- Edge deployment
- 빠른 응답 필요 (chatbot)
```

Deepseek-V2가 적합한 경우

```
1. 대형 모델 (> 100B params)
2. 높은 parameter efficiency 필요
3. 많은 expert 필요 (32+개)
4. 복잡한 태스크 전문화
```

```
예시:
- 대규모 언어 모델 (200B+)
- 다양한 도메인 커버
- 연구/생산 환경
- 최고 성능 요구 (코딩, 수학, 추론)
```

6.7 성능 비교 (이론적)

Metric	Mixtral-8x7B	Deepseek-V2-236B
Inference Speed	⭐⭐⭐⭐⭐ (빠름)	⭐⭐⭐⭐ (약간 느림)
Memory Efficiency	⭐⭐⭐⭐ (좋음)	⭐⭐⭐⭐⭐ (매우 좋음)
Routing Overhead	⭐⭐⭐⭐⭐ (낮음)	⭐⭐⭐⭐ (약간 높음)
Expert Utilization	⭐⭐⭐ (보통)	⭐⭐⭐⭐⭐ (매우 좋음)
Load Balancing	⭐⭐⭐ (보통)	⭐⭐⭐⭐⭐ (매우 좋음)
Scalability	⭐⭐⭐ (제한적)	⭐⭐⭐⭐⭐ (우수)
Implementation	⭐⭐⭐⭐⭐ (간단)	⭐⭐⭐ (복잡)

종합:

- **Mixtral**: 작고 빠른 모델에 최적
- **Deepseek-V2**: 대형 모델, 높은 효율성 필요시 최적

6.8 코드 비교

Mixtral Forward

```python
def mixtral_moe_forward(hidden_states):
    # 1. Router
    router_logits, _ = self.gate(hidden_states)

    # 2. FusedMoE (Standard Top-K)
    output = self.experts(
        hidden_states=hidden_states,
        router_logits=router_logits,
    )
```

```
    return output

# 단순! 3줄로 끝
```

Deepseek-V2 Forward

```
def deepseek_v2_moe_forward(hidden_states):
    # 1. Router
    router_logits, _ = self.gate(hidden_states)

    # 2. SharedFusedMoE (Shared + Grouped Top-K)
    shared_output, routed_output = self.experts(
        hidden_states=hidden_states,
        router_logits=router_logits,
    )

    # 3. Scaling
    routed_output *= self.routed_scaling_factor

    # 4. Combination
    output = shared_output + routed_output

    return output

# 더 복잡! Shared + Routing + Scaling
```

6.9 선택 가이드

```
모델 크기가 작다 (< 50B)
  └▶ Mixtral
       └▶ Standard Top-K로 충분
       └▶ 빠른 inference

모델 크기가 크다 (> 100B)
  └▶ Deepseek-V2
       └▶ Grouped Top-K 필요
       └▶ Shared experts로 안정성
       └▶ 높은 efficiency

Expert 수가 적다 (8-16개)
  └▶ Mixtral
       └▶ Standard Top-K 효율적
```

```
Expert 수가 많다 (32+개)
  └▶ Deepseek-V2
       └▶ Grouped routing 필수
       └▶ Load balancing 개선

빠른 개발/단순성 중요
  └▶ Mixtral
       └▶ 구현 간단

최고 성능/효율성 중요
  └▶ Deepseek-V2
       └▶ 11x efficiency
```

---

7. Expert Parallelism & Load Balancing

MoE 모델의 실제 배포에서는 **Expert Parallelism (EP)**을 사용하여 experts를 여러 GPU에 분산합니다.

7.1 Expert Parallelism (EP)

```
# Tensor Parallelism (TP): weight를 나눠서 분산
# Expert Parallelism (EP): expert를 나눠서 분산

# 예시: Mixtral-8x7B on 4 GPUs with EP
GPU 0: Expert 0, Expert 1  (2 experts)
GPU 1: Expert 2, Expert 3  (2 experts)
GPU 2: Expert 4, Expert 5  (2 experts)
GPU 3: Expert 6, Expert 7  (2 experts)

# Token routing with EP:
Token 0 → Expert 3 (GPU 1), Expert 5 (GPU 2)
  → All-to-All communication 필요!
```

장점:

- Expert weights를 여러 GPU에 분산 → 메모리 부담 감소
- 각 GPU가 일부 expert만 관리 → 효율적

단점:

- All-to-All communication 오버헤드
- Network bandwidth가 병목될 수 있음

7.2 Load Balancing

문제: 특정 expert에 토큰이 몰림

```
# Bad distribution:
Expert 0: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ 40% (과부하!)
Expert 1: ▓▓ 5%
Expert 2: ▓▓▓▓▓▓▓▓▓ 15%
Expert 3: ▓▓▓▓▓▓▓ 10%
Expert 4: ▓▓▓▓ 8%
Expert 5: ▓▓▓▓▓▓▓ 12%
Expert 6: ▓▓▓▓ 7%
Expert 7: ▓ 3%

# GPU 0 (Expert 0, 1): 45% load
# GPU 1 (Expert 2, 3): 25% load  ← Imbalance!
# GPU 2 (Expert 4, 5): 20% load
# GPU 3 (Expert 6, 7): 10% load
```

해결책:

1. **Load Balancing Loss**:

```
# Router 학습 시 추가 loss
balance_loss = torch.var(expert_usage)
total_loss = task_loss + α × balance_loss
```

2. **Expert Capacity**:

```
# 각 expert가 처리할 수 있는 최대 토큰 수 제한
capacity = (num_tokens × top_k / num_experts) × capacity_factor
# capacity_factor = 1.25 (25% buffer)
```

3. **Grouped Routing** (Deepseek-V2):

```
# 각 그룹에서 1개씩 선택 → 자동으로 분산
```

8. 성능 측정 및 최적화

8.1 vLLM에서의 MoE 성능

Mixtral-8x7B 성능 (vLLM 기준):

```
# Hardware: 2× A100 80GB

Throughput:
  - Batch size 1:   ~15 tokens/sec per request
  - Batch size 32:  ~450 tokens/sec total
  - Batch size 64:  ~800 tokens/sec total

Latency:
  - First token (TTFT):  ~150ms
  - Inter-token:          ~35ms

Memory:
  - Model weights:  ~95GB (FP16)
  - KV cache:       ~15GB (batch 32, 2048 ctx)
  - Peak:           ~110GB

Utilization:
  - GPU:      75-85%
  - Memory:   80-90%
```

8.2 최적화 기법

(1) FusedMoE Kernel

```
# vLLM의 Triton kernel 최적화:
# - Weight fusion: 모든 expert를 하나의 tensor로
# - Token sorting: Expert별 배치 처리
# - Atomic add: 결과 효율적 합침

# Speedup: 25x vs naive implementation
```

(2) Expert Parallelism

```
# EP size 선택:
# - EP=1: 모든 expert가 각 GPU에 (메모리 많이 필요)
# - EP=2: Expert를 2 GPU에 분산
# - EP=4: Expert를 4 GPU에 분산 (통신 오버헤드)

# 권장:
# Mixtral-8x7B → EP=2 (A100 80GB 2장)
# Deepseek-V2  → EP=8 (A100 80GB 8장)
```

(3) KV Cache 관리

```
# vLLM의 PagedAttention:
# - MoE와 결합하여 메모리 효율 극대화
# - Block 단위로 KV cache 관리

# 예시:
# Without PagedAttention: 150GB memory
# With PagedAttention:    110GB memory
# → 27% memory saving
```

---

9. 트러블슈팅

9.1 Out of Memory (OOM)

증상:

```
RuntimeError: CUDA out of memory
```

원인 및 해결:

1. **Expert weights가 너무 큼**:

```
# 해결책 1: Quantization
# FP16 → INT8 또는 INT4
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mixtral-8x7B-v0.1",
    load_in_8bit=True,  # 8-bit quantization
)
```

```
# 메모리 절감: ~50%
```

2. **EP size 증가**:

```
# 더 많은 GPU에 expert 분산
vllm serve mixtralai/Mixtral-8x7B-v0.1 \
  --tensor-parallel-size 2 \
  --expert-parallel-size 2  # EP=2로 증가
```

3. **Batch size 감소**:

```
# 배치 크기를 줄여서 KV cache 메모리 절약
--max-num-seqs 16  # 32 → 16으로 감소
```

9.2 느린 Inference

증상:

```
Throughput: 50 tokens/sec (예상: 450 tokens/sec)
```

원인 및 해결:

1. **EP communication 병목**:

```
# EP size를 줄여서 통신 오버헤드 감소
# EP=8 → EP=4 또는 EP=2

# 또는 faster interconnect 사용 (NVLink, InfiniBand)
```

2. **작은 batch size**:

```
# Batch size를 늘려서 GPU 활용률 증가
--max-num-seqs 64  # 16 → 64로 증가
--max-num-batched-tokens 8192
```

3. **Load imbalance**:

```
# Expert 사용률 확인:
# → 일부 expert에 토큰 몰림
```

```
# 해결: Grouped routing 또는 load balancing loss
```

9.3 Expert 미사용

증상:

```
Expert 0: 35% utilization
Expert 1: 2%  ← 거의 사용 안 됨!
Expert 2: 18%
```

원인:

- Router가 특정 expert만 선택
- 학습 데이터 불균형
- Routing weights 초기화 문제

해결:

```
# 1. 모델 재학습 (load balancing loss 추가)

# 2. Grouped routing 사용 (Deepseek-V2)

# 3. Expert dropout 적용
```

9.4 Numerical Instability

증상:

```
RuntimeError: NaN in output
```

원인 및 해결:

```
# FP16 overflow in routed experts

# 해결책 1: Scaling factor (Deepseek-V2 방식)
routed_output *= 0.125  # Scale down

# 해결책 2: Mixed precision
# FP16 for weights, FP32 for accumulation
```

```
# 해결책 3: Gradient clipping
clip_grad_norm_(model.parameters(), max_norm=1.0)
```

---

10. 요약

10.1 핵심 포인트

MoE의 핵심:

1. ✅ **Sparse Activation**: 일부 expert만 활성화 → 효율성
2. ✅ **Router**: 각 토큰에 적합한 expert 선택
3. ✅ **FusedMoE**: Triton kernel로 병렬 실행 → 성능
4. ✅ **Parameter Efficiency**: Active params ≪ Total params

Mixtral:

- Standard Top-K routing
- 8 experts, Top-2
- 간단하고 빠름
- 5x efficiency

Deepseek-V2:

- Shared experts + Grouped Top-K
- 64 routed experts + 2 shared
- 복잡하지만 효율적
- 11x efficiency

10.2 실무 권장사항

모델 선택:

```
if model_size < 50B:
    use_mixtral()  # Simple and fast
elif model_size > 100B:
    use_deepseek_v2()  # High efficiency
```

배포 설정:

```
# Mixtral-8x7B on 2× A100 80GB:
vllm serve mistralai/Mixtral-8x7B-v0.1 \
  --tensor-parallel-size 2 \
  --max-num-seqs 32 \
  --gpu-memory-utilization 0.90

# Deepseek-V2 on 8× A100 80GB:
vllm serve deepseek-ai/DeepSeek-V2 \
  --tensor-parallel-size 8 \
  --expert-parallel-size 8 \
  --max-num-seqs 64 \
  --gpu-memory-utilization 0.85
```

최적화 우선순위:

1. FusedMoE kernel 활성화 (vLLM 기본)
2. 적절한 EP size 선택
3. Batch size 최적화
4. KV cache 관리 (PagedAttention)
5. Quantization 고려 (메모리 부족 시)

10.3 참고 자료

논문:

- Mixtral: "Mixtral of Experts" (Mistral AI, 2024)
- Deepseek-V2: "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model" (2024)
- MoE Survey: "Mixture-of-Experts Meets Instruction Tuning" (2023)

vLLM 코드 위치:

- Mixtral: `vllm/model_executor/models/mixtral.py`
- Deepseek-V2: `vllm/model_executor/models/deepseek_v2.py`
- FusedMoE: `vllm/model_executor/layers/fused_moe/`
- MoE kernels: `vllm/model_executor/layers/fused_moe/fused_moe.py`

관련 문서:

- [01. Weight Loading]

- 02. Model Initialization
- 03. Inference Process
- 04. PagedAttention
- 05. Transformer LLMs

문서 작성 완료! 🎉

이 문서에서 다룬 내용:

1. MoE 기본 개념과 장단점
2. Mixtral 아키텍처 및 구현 상세
3. MixtralMoE 클래스 및 Router 분석
4. FusedMoE Triton kernel 최적화
5. Deepseek-V2 MoE (Shared Experts + Grouped Top-K)
6. Mixtral vs Deepseek-V2 비교
7. Expert Parallelism & Load Balancing
8. 성능 측정 및 최적화
9. 트러블슈팅 가이드

MoE LLMs의 모든 핵심 개념을 vLLM 코드 기반으로 상세히 분석했습니다!